



Course Title:

CS 342 Operating System

Submitted to:

Dr. Zakia

**Drone Charging Station
Management System**

Submitted by:

4864-Foc/Bscs/F23b Khadeeja Taj

4856-Foc/Bscs/F23b Laiba Sabir

4836-Foc/Bscs/F23b Najia Zainab

4873-Foc/Bscs/F23b Manahil khan

Department of Computer Science

International Islamic University, Islamabad

Contents

1. Introduction.....	2
2. Problem Statement	2
3. Objectives	2
4. System Scenario.....	3
5. System Design	3
Main Components:	3
Queue Design:.....	3
Charging Time Quantum:.....	4
6. Multithreading.....	4
Thread Lifecycle:	4
7. Producer Consumer Concept	4
Producer Consumer Workflow:	4
8. Multilevel Feedback Queue (MLFQ) Scheduling.....	5
MLFQ Rules:	5
9. Semaphore and Mutex Synchronization	5
Why Synchronization Is Important:	6
10. Shared Resource Management.....	6
Station Array:.....	6
Resource Lifecycle:	6
11. Starvation Prevention Using Aging.....	6
Promotion Rules:.....	7
12. Code	7
13. Output Screenshots	Error! Bookmark not defined.
14. Conclusion	19
15. References.....	19

Drone Charging Station Management System

1. Introduction

The Drone Charging Station Management System is a simulation-based Operating Systems project developed to demonstrate important OS concepts through a real-world application. The system simulates multiple drones requesting access to limited charging stations while being managed using scheduling and synchronization techniques.

Each drone operates as an independent thread and dynamically requests charging access. Charging stations are treated as limited shared resources and are controlled using semaphores and mutex synchronization. The system uses the Multi-Level Feedback Queue (MLFQ) scheduling algorithm to prioritize emergency drones while ensuring fairness through starvation prevention using aging.

This project demonstrates practical implementation of:

- Multithreading
- Producer Consumer Concept
- MLFQ Scheduling
- Semaphores and Mutex Synchronization
- Shared Resource Management
- Starvation Prevention

The project provides a realistic understanding of synchronization and scheduling problems found in modern smart delivery systems.

2. Problem Statement

A smart delivery company uses multiple drones to deliver medical supplies, food, and parcels across a city. All drones must periodically access limited charging stations before continuing their deliveries. The company faces several issues:

- Emergency drones should receive charging access faster.
- Low-priority drones may suffer from starvation.
- Multiple drones may try to access charging stations simultaneously.
- Improper scheduling may increase waiting time and reduce efficiency.

The goal of this project is to design and implement a Drone Charging Station Management System using Operating System concepts. The system must simulate:

- Drone charging requests
- Queue scheduling
- Charging station access
- Charging completion
- Delivery resumption

3. Objectives

The project specifically demonstrates:

- Multilevel Feedback Queue (MLFQ) Scheduling
- Multithreading
- Semaphores and Mutex Synchronization
- Starvation Prevention
- Shared Resource Management
- Producer Consumer Concept

Additional objectives include:

- Efficient drone scheduling
- Proper synchronization of shared resources
- Fair charging station allocation
- Reduced waiting time
- Prevention of race conditions and starvation

4. System Scenario

The Drone Charging Station Management System works as follows:

- Each drone acts as a separate thread.
- Delivery requests arrive dynamically.
- Drones enter different priority queues.
- Emergency drones receive higher priority.
- Charging stations are limited shared resources.
- Semaphores control station access.
- Mutex locks protect shared queues and station arrays.
- Aging prevents starvation of low-priority drones.

5. System Design

Main Components:

Component	Purpose
Drone Threads	Simulate drones operating independently
Producer Thread	Generates delivery requests dynamically
MLFQ Queues	Manage scheduling priorities
Charging Stations	Shared charging resources
Semaphore	Controls station availability
Mutex	Protects shared data
Aging Thread	Prevents starvation

Queue Design:

Queue	Drone Type	Priority
Q0	Emergency	Highest
Q1	Medical	High
Q2	Food	Medium
Q3	Parcel	Low

Charging Time Quantum:

Queue	Charging Time
Q0	5 seconds
Q1	4 seconds
Q2	3 seconds
Q3	2 seconds

6. Multithreading

A thread is the smallest unit of execution inside a process. Threads allow multiple tasks to execute simultaneously. In this project, every drone is implemented as a separate thread. This allows multiple drones to:

- Wait for charging
- Charge batteries
- Resume delivery
- Operate concurrently

Multithreading improves realism and system efficiency.

Thread Lifecycle:

Each drone thread performs the following operations:

1. Request charging access
2. Enter scheduling queue
3. Wait for available station
4. Charge battery
5. Resume delivery

7. Producer Consumer Concept

The Producer Consumer problem is a classical synchronization problem in Operating Systems.

- Producer generates data
- Consumer processes dataShared buffer stores requests

In this project:

Producer	Consumer
producer_thread()	drone_thread()

The producer dynamically generates delivery requests while drone threads consume and process them.

Producer Consumer Workflow:

- a. Producer creates request
- b. Request enters queue
- c. Scheduler selects drone

- d. Drone consumes request
- e. Drone charges battery
- f. Drone resumes delivery

8. Multilevel Feedback Queue (MLFQ) Scheduling

Multilevel Feedback Queue (MLFQ) Scheduling is an advanced scheduling algorithm that uses multiple queues with different priorities. The scheduler always selects drones from the highest-priority non-empty queue. This allows:

- Emergency drones to charge first
- Better responsiveness
- Fair scheduling
- Reduced waiting time

MLFQ Rules:

Rule	Description
Rule 1	Higher-priority drones run first
Rule 2	Same-priority drones share resources fairly
Rule 3	New drones enter appropriate queue
Rule 4	Lower queues get smaller time quantum
Rule 5	Aging promotes long-waiting drones

9. Semaphore and Mutex Synchronization

Synchronization prevents race conditions when multiple threads access shared resources simultaneously. This project uses:

- Semaphores
- Mutex Locks

Semaphore:

Semaphores control access to the limited charging stations.

The system has only 3 charging stations:

```
sem_init(&charging_stations, 0, NUM_STATIONS);
```

Before charging:

```
sem_wait(&charging_stations);
```

After charging:

```
sem_post(&charging_stations);
```

This ensures that only 3 drones can charge simultaneously.

Mutex:

Mutex locks protect shared data structures.

Used for:

- Queue protection
- Station array protection

Example:

- `pthread_mutex_lock(&queue_mutex);`
- `pthread_mutex_unlock(&queue_mutex);`

Why Synchronization Is Important:

Synchronization prevents:

- Race conditions
- Data corruption
- Multiple drones using same station
- Inconsistent queue states

10.Shared Resource Management

A shared resource is any resource accessed by multiple concurrent threads. **Charging Stations** are shared resources.

Station Array:

```
int stations[NUM_STATIONS] = {0};
```

Each station may be:

- FREE
- Occupied by a drone

Resource Lifecycle:

1. Drone requests charging
2. Drone waits for station
3. Semaphore grants access
4. Station assigned
5. Drone charges battery
6. Station released
7. Next drone acquires station

11.Starvation Prevention Using Aging

Starvation occurs when low-priority drones wait indefinitely because higher-priority drones continuously receive charging access.To solve this problem, the system implements Aging. Aging gradually increases the priority of drones waiting too long in lower-priority queues.

Promotion Rules:

From Queue	To Queue	Waiting Time
Q3	Q2	10 seconds
Q2	Q1	10 seconds

12.Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
#define NUM_DRONES 8
```

```
#define NUM_STATIONS 3
```

```
#define NUM_QUEUES 4
```

```
#define MAX_DELIVERIES 3
```

```
#define QUEUE_SIZE 20
```

//DRONE STRUCT

```
typedef struct {
```

```
    int id;
```

```
    char type[20];
```

```
    int priority;
```

```
    int wait_time;
```

```
    int queue_level;
```

```
} Drone;
```


//GLOBAL VARIABLES

```
Drone q0[QUEUE_SIZE];

Drone q1[QUEUE_SIZE];

Drone q2[QUEUE_SIZE];

Drone q3[QUEUE_SIZE];

int q0_count = 0, q1_count = 0, q2_count = 0, q3_count = 0;


pthread_mutex_t queue_mutex;

pthread_mutex_t station_mutex;

pthread_mutex_t print_mutex;

sem_t charging_stations;


int stations[NUM_STATIONS] = {0};

volatile int simulation_running = 1;

volatile int drones_done = 0;

pthread_mutex_t done_mutex = PTHREAD_MUTEX_INITIALIZER;
```

//PRINT QUEUE STATE

```
void print_queue_state() {

    printf(" [QUEUES] Q0-Emergency:%d Q1-Medical:%d Q2-Food:%d Q3-Parcel:%d\n",q0_count,
q1_count, q2_count, q3_count);

}
```

//ENQUEUE DRONE

```
void enqueue_drone(Drone d) {

    pthread_mutex_lock(&queue_mutex);

    if(d.priority == 1) {

        q0[q0_count++] = d;
```

```

}

else if (d.priority == 2) {

q1[q1_count++] = d;

}

else if (d.priority == 3) {

q2[q2_count++] = d;

}

else {

q3[q3_count++] = d;

}


printf(" [ENQUEUE] Drone #%02d %-9s --> Q%d\n",d.id, d.type, d.priority - 1);

print_queue_state();

pthread_mutex_unlock(&queue_mutex);

}

```

//SCHEDULER

```

Drone schedule_next_drone() {

    Drone selected;

    selected.id = -1;

    pthread_mutex_lock(&queue_mutex);

    if (q0_count > 0) {

        selected = q0[0];

        for (int i = 0; i < q0_count - 1; i++)

            q0[i] = q0[i+1];

        q0_count--;

    } else if (q1_count > 0) {

```

```

    selected = q1[0];

    for (int i = 0; i < q1_count - 1; i++)

        q1[i] = q1[i+1];

        q1_count--;

} else if (q2_count > 0) {

    selected = q2[0];

    for (int i = 0; i < q2_count - 1; i++)

        q2[i] = q2[i+1];

        q2_count--;

} else if (q3_count > 0) {

    selected = q3[0];

    for (int i = 0; i < q3_count - 1; i++)

        q3[i] = q3[i+1];

        q3_count--;

}

pthread_mutex_unlock(&queue_mutex);

return selected;

}

```

//PRODUCER THREAD

```

void* producer_thread(void* arg) {

    (void)arg;

    srand(time(NULL));

    int request_id = 1;

    while (simulation_running) {

        sleep(rand() % 3 + 1);
    }
}

```

```

if (!simulation_running)

    break;

Drone d;

d.id = request_id++;

d.wait_time= 0;

d.queue_level= 0;


int t = rand() % 4;

if(t == 0) {

strcpy(d.type, "EMERGENCY");

d.priority = 1;

}

else if (t == 1) {

strcpy(d.type, "MEDICAL");

d.priority = 2;

}

else if (t == 2) {

strcpy(d.type, "FOOD");

d.priority = 3;

}

else{

strcpy(d.type, "PARCEL");

d.priority = 4;

}

pthread_mutex_lock(&print_mutex);

printf(" -----\n");

```

```

    printf(" [PRODUCER] New request --> Drone #%02d (%s)\n", d.id, d.type);

    enqueue_drone(d);

    pthread_mutex_unlock(&print_mutex);

}

pthread_mutex_lock(&print_mutex);

printf(" ----- \n");

printf(" [PRODUCER] Shutting down\n");

pthread_mutex_unlock(&print_mutex);

return NULL;

}

```

//DRONE THREAD

```

void* drone_thread(void* arg) {

    Drone* drone = (Drone*)arg;

    pthread_mutex_lock(&print_mutex);

    printf(" [THREAD] Drone #%02d started\n", drone->id);

    pthread_mutex_unlock(&print_mutex);


    for (int cycle = 0; cycle < MAX_DELIVERIES; cycle++) {

        //wait for a request

        Drone current;

        current.id = -1;

        while (current.id == -1 && simulation_running) {

            current = schedule_next_drone();

            if (current.id == -1)

                sleep(1);
        }
    }
}

```

```

}

if (!simulation_running)

break;

//waiting for station

pthread_mutex_lock(&print_mutex);

printf(" -----\\n");

printf(" [SYNC]   Drone #%02d %-9s waiting for station...\\n",current.id, current.type);

pthread_mutex_unlock(&print_mutex);

sem_wait(&charging_stations);


//acquire a free station

int station_id = -1;

pthread_mutex_lock(&station_mutex);

for (int i = 0; i < NUM_STATIONS; i++) {

    if (stations[i] == 0) {

        stations[i] = current.id;

        station_id = i;

        break;

    }

}

pthread_mutex_unlock(&station_mutex);

//charging

int charge_time;

if (current.priority == 1)

charge_time = 5;

else if (current.priority == 2)

```

```

    charge_time = 4;

    else if (current.priority == 3)

charge_time = 3;

    else

    charge_time = 2;

    pthread_mutex_lock(&print_mutex);

    printf(" [STATION] Drone #02d %-9s --> Station %d acquired\n",current.id, current.type, station_id +
1);

    printf(" [CHARGING] Drone #02d %-9s charging for %ds...\n",current.id, current.type, charge_time);

    pthread_mutex_unlock(&print_mutex);


    sleep(charge_time);


//release station

    pthread_mutex_lock(&station_mutex);

    stations[station_id] = 0;

    pthread_mutex_unlock(&station_mutex);


    sem_post(&charging_stations);

    pthread_mutex_lock(&print_mutex);

    printf(" [DONE] Drone #02d %-9s --> Station %d released | delivery resumed\n",current.id,
current.type, station_id + 1);

    pthread_mutex_unlock(&print_mutex);

    sleep(1);

}

pthread_mutex_lock(&print_mutex);

printf(" ----- \n");

```

```

printf(" [COMPLETE] Drone #%%02d finished all %d deliveries\n",drone->id, MAX_DELIVERIES);

pthread_mutex_unlock(&print_mutex);


pthread_mutex_lock(&done_mutex);

drones_done++;

if (drones_done == NUM_DRONES) {

    simulation_running = 0;

    pthread_mutex_lock(&print_mutex);

    printf(" =====\n");

    printf(" [SYSTEM] All %d drones finished. Shutting down...\n", NUM_DRONES);

    printf(" =====\n");

    pthread_mutex_unlock(&print_mutex);

}

pthread_mutex_unlock(&done_mutex);

return NULL;

}

```

//AGING THREAD

```

void* aging_thread(void* arg) {

    (void)arg;

    while (simulation_running) {

        sleep(5);

        if (!simulation_running)

            break;

        pthread_mutex_lock(&queue_mutex);

        /* Q3 -> Q2 */

        for (int i = 0; i < q3_count; i++) {

```



```

    q3[i].wait_time += 5;

    if (q3[i].wait_time >= 10) {

        pthread_mutex_lock(&print_mutex);

        printf(" -----\\n");

        printf(" [AGING]   Drone #%02d waited %ds in Q3 --> promoted to Q2\\n",q3[i].id,
q3[i].wait_time);

        pthread_mutex_unlock(&print_mutex);

        q3[i].wait_time = 0;

        q2[q2_count++] = q3[i];

        for (int j = i; j < q3_count - 1; j++)

            q3[j] = q3[j+1];

        q3_count--;

        i--;

    }

}

/* Q2 -> Q1 */

for (int i = 0; i < q2_count; i++) {

    q2[i].wait_time += 5;

    if (q2[i].wait_time >= 10) {

        pthread_mutex_lock(&print_mutex);

        printf(" -----\\n");

        printf(" [AGING]   Drone #%02d waited %ds in Q2 --> promoted to Q1\\n",q2[i].id,
q2[i].wait_time);

        pthread_mutex_unlock(&print_mutex);

        q2[i].wait_time = 0;

        q1[q1_count++] = q2[i];

        for (int j = i; j < q2_count - 1; j++)

```

```

        q2[j] = q2[j+1];

        q2_count--;

        i--;
    }

}

pthread_mutex_unlock(&queue_mutex);

}

return NULL;

}

//MAIN

int main() {

    pthread_mutex_init(&print_mutex,NULL);

    pthread_mutex_init(&queue_mutex,NULL);

    pthread_mutex_init(&station_mutex,NULL);

    sem_init(&charging_stations, 0, NUM_STATIONS);

    printf("=====\n");

    printf("    DRONE CHARGING STATION MANAGEMENT SYSTEM\n");

    printf(" \nDrones: %d | Stations: %d | Deliveries per drone: %d\n",NUM_DRONES, NUM_STATIONS,
MAX_DELIVERIES);

    printf(" Queues: Q0=Emergency Q1=Medical Q2=Food Q3=Parcel\n");

    printf(" Charge: Q0=5s Q1=4s Q2=3s Q3=2s\n");

    printf(" Aging: Q3->Q2 after 10s | Q2->Q1 after 10s\n");

    printf(" =====\n");

    printf("\n");

    Drone drones[NUM_DRONES];

```

```

for (int i = 0; i < NUM_DRONES; i++) {

    drones[i].id = i + 1;

    drones[i].wait_time= 0;

    drones[i].queue_level = 0;

}

pthread_t producer, aging, drone_threads[NUM_DRONES];

pthread_create(&producer, NULL, producer_thread, NULL);

pthread_create(&aging,NULL, aging_thread,NULL);

for (int i = 0; i < NUM_DRONES; i++)

    pthread_create(&drone_threads[i], NULL, drone_thread, &drones[i]);

for (int i = 0; i < NUM_DRONES; i++)

    pthread_join(drone_threads[i], NULL);


pthread_join(producer, NULL);

pthread_join(aging,NULL);


sem_destroy(&charging_stations);

pthread_mutex_destroy(&queue_mutex);

pthread_mutex_destroy(&station_mutex);

pthread_mutex_destroy(&print_mutex);

pthread_mutex_destroy(&done_mutex);

printf(" =====\n");

printf("      SIMULATION COMPLETE\n");

printf(" =====\n");

return 0;

}

```

13. Conclusion

The Drone Charging Station Management System successfully demonstrates major Operating System concepts through a real-world simulation. The project implements:

- Multithreading
- Producer Consumer Architecture
- MLFQ Scheduling
- Semaphore Synchronization
- Mutex Synchronization
- Shared Resource Management
- Starvation Prevention

The system efficiently manages multiple drones requesting limited charging stations while maintaining synchronization and fairness. The aging mechanism prevents starvation by promoting drones waiting too long in lower-priority queues. This project provides practical understanding of modern Operating System scheduling and synchronization techniques.

14. References

1. Operating System Concepts — Silberschatz
2. POSIX pthread documentation
3. Linux semaphore documentation
4. GeeksforGeeks Operating System Articles
5. MLFQ Scheduling Tutorials
6. Multithreading and Synchronization Tutorials